Quiz 11

Tick the statements that are true and leave unticked those that are false.
Each statement is worth 1 point for a correct answer.

Your name: _____

- ☒ A polynomial-time Turing machine can visit only polynomially many cells on its work tape.
- ☐ If a Turing machine uses polynomial space, it must halt within polynomially many steps.
- ☒ If a Turing machine uses polynomial space, it must halt within exponentially many steps.
- ☒ It is an open question if PSPACE $\subseteq$ P.
- ☐ It is an open question if PSPACE $\subset$ P.
- ☒ A Turing machine that decides a language using logarithmic space also runs in polynomial time.
- ☒ L $\subset$ PSPACE, strictly.
- ☐ If a Turing machine M decides a language $A \in$ L, then it uses space $O(\log n)$.
- ☐ A nondeterministic machine can decide HALT by guessing whether the input program halts.
- ☐ Nondeterministic machines are a formal model of the unpredictable behaviour of Windows 11.

Bonus. A language B is P-complete under polynomial-time reductions if $B \in$ P and every language $A \in$ P satisfies $A \leq_p B$. Which languages in P are P-complete under this definition? Justify your answer.

Possible answer: Exactly the languages $B \in$ P with $B \neq \emptyset$ and $B \neq \Sigma^*$. If B is nontrivial, choose $b \in B$ and $c \in \Sigma^* \setminus B$. For any $A \in$ P, the map r with $r(w) = b$ for $w \in A$ and $r(w) = c$ otherwise is computable in polynomial time and satisfies

$$w \in A \iff r(w) \in B,$$

so $A \leq_p B$. Conversely, $\emptyset$ is not P-complete: for $A = \Sigma^* \in$ P, no reduction can send a yes-instance into $\emptyset$. Similarly, Σ^* is not P-complete: for $A = \emptyset \in$ P, every output belongs to Σ^*, so no reduction preserves the answer. Thus exactly the nontrivial languages in P are P-complete.

Explanations

☒ *A polynomial-time Turing machine can visit only polynomially many cells on its work tape.* If M runs in time $p(n)$ for a polynomial p, it performs at most $p(n)$ steps. Each step can move the work-tape head by at most one cell, so it can visit at most $p(n) + 1$ cells: the starting cell and at most one new cell per step. Thus the machine uses polynomial space.

☐ *If a Turing machine uses polynomial space, it must halt within polynomially many steps.* Space can be reused many times. For example, on an input of length n, a machine can use n work cells as a binary counter and count from 0 to 2^n before halting; it uses $O(n)$ space but 2^n steps.

☒ *If a Turing machine uses polynomial space, it must halt within exponentially many steps.* By our definition, a machine using space $f(n)$ does not even have to halt—it only has to avoid visiting more than $O(f(n))$ cells on an input of length n. Thus the statement is false as written: a machine can loop forever while using constant, hence polynomial, space. If a machine uses polynomial space and *always halts*, then configuration counting shows that it halts within exponentially many steps. Some textbooks define 'space usage' for deciders only, i.e. for machines that always halt by definition. This may have caused confusion, so both answers to this statement are accepted as correct; the half-cross marks this convention.

☒ *It is an open question if* $\mathrm{PSPACE} \subseteq \mathrm{P}$. Since $\mathrm{P} \subseteq \mathrm{PSPACE}$, $\mathrm{PSPACE} \subseteq \mathrm{P}$ is equivalent to $\mathrm{P} = \mathrm{PSPACE}$, which is open.

☐ *It is an open question if* $\mathrm{PSPACE} \subset \mathrm{P}$. This is not open: $\mathrm{P} \subseteq \mathrm{PSPACE}$ is known, so $\mathrm{PSPACE} \subset \mathrm{P}$ is impossible.

☒ *A Turing machine that decides a language using logarithmic space also runs in polynomial time.* A machine that decides a language always halts. So the maximal number of steps is exponential in terms of used space. For logarithmic space, it is thus polynomially many.

☒ $\mathrm{L} \subset \mathrm{PSPACE}$*, strictly.* We have $\mathrm{L} = \mathrm{SPACE}(\log n) \subseteq \mathrm{SPACE}(n)$. By the space hierarchy theorem, $\mathrm{SPACE}(n) \subset \mathrm{SPACE}\left(n^2\right)$, and $\mathrm{SPACE}\left(n^2\right) \subseteq \mathrm{PSPACE}$. Therefore $\mathrm{L} \subset \mathrm{PSPACE}$, strictly.

☐ *If a Turing machine M decides a language $A \in \mathrm{L}$, then it uses space $O(\log n)$.* Membership $A \in \mathrm{L}$ means that *some* machine decides A using $O(\log n)$ space; it does not mean that every machine deciding A uses that little space. A decider for A can first mark n^2 work-tape cells and then run the log-space algorithm, so it still decides A but uses $O(n^2)$ space.

☐ *A nondeterministic machine can decide* HALT *by guessing whether the input program halts.* Nondeterminism does not make undecidable problems decidable. If a nondeterministic machine decided HALT, simulating all its branches deterministically, for example by breadth-first search, would give a deterministic decider for HALT. Moreover, a guess cannot certify non-halting: a branch that guesses "halts" has no way to verify that the program really halts.

☐ *Nondeterministic machines are a formal model of the unpredictable behaviour of Windows 11.* Nondeterminism is a mathematical model of existential choice: a computation accepts if at least one branch accepts, and every branch must halt. It is not a model of random or unpredictable events.